

Aula 8 - Pilhas, filas e vetores

Até hoje, a única estrutura de dados que conhecemos são os vetores: estruturas sequenciais de dados extremamente úteis. O objetivo dessa aula é, além de revisar brevemente os vetores, aprendermos também sobre as estruturas de pilhas e filas.

Vector

Declaramos um vector da seguinte forma:

```
vector<tipo> nome(tamanho_opcional, valor_padrao_opcional);
```

Os vectors possuem uma função de inserção de dados, e uma de remoção de dados, respectivamente:

```
vetor.push_back(elemento);  
vetor.pop_back();
```

E, conseguimos acessar informações dentro de um vetor livremente:

```
cout << vetor[posicao_qualquer];
```

Pilha (stack)

As pilhas são estruturas de dados de tamanho dinâmico, similar aos vectors, mas, nelas, não temos a opção de declarar um tamanho prévio. Declaramos uma pilha da seguinte forma:

```
stack<tipo> pilha;
```

As pilhas também possuem uma função de inserção de dados:

```
pilha.push(elemento);
```

E uma de remoção de dados:

```
pilha.pop();
```

Por fim, para acessar informações dentro de uma pilha, a única opção que temos é a de acessar a informação no **topo** da pilha, usando a função abaixo:

```
cout << "O elemento no topo da pilha é: " << pilha.top();
```

O funcionamento da pilha segue a filosofia “filo” (uma nomenclatura que vocês vão escutar muito durante a faculdade) que significa “first in, last out”.

Basicamente, ela funciona exatamente como você imaginaria que “uma pilha de informações” funcionaria, literalmente.

Ao adicionar dados, você empilha eles, de forma que o único dado que você consegue acessar é aquele que está no topo (o que você empilhou por último). O primeiro dado empilhado fica enterrado, e você só consegue remover ele depois de ter removido todos os outros dados em cima dele.

Notem que a função `top()` apenas consulta a informação no topo da pilha, sem removê-la. Se vocês quiserem **consultar E remover** usem a `.top()` seguida da `.pop()`.

Fila (queue)

Também são estruturas de dados dinâmicas, que não conseguimos declarar um tamanho prévio. Sua sintaxe é bem parecida com a da pilha:

```
queue<tipo> fila;
```

Também temos uma função `push` idêntica à da pilha:

```
fila.push(elemento);
```

E uma `pop` igual:

```
fila.pop();
```

A função de consulta de dados é diferente. Aqui, temos a opção de olhar para o começo da fila e para o final dela:

```
cout << "O primeiro elemento da fila é " << fila.front();  
cout << "O último é: " << fila.back();
```

A fila funciona, também, exatamente como você está imaginando. O cara que chegou primeiro é também o primeiro a sair. Isso é a filosofia “fifo”, ou seja “first in, first out”.

Funções comuns às duas

Outras funções úteis que ambas possuem são as seguintes:

```
fila.empty(); // vale true se a fila/pilha estiver vazia e fa  
lse caso contrário  
fila.size(); // retorna o tamanho da fila/pilha
```

E, para usá-las, se você não estiver usando a `bits/stdc++.h`:

```
#include <iostream>  
#include <stack>  
#include <queue>  
using namespace std;  
  
int main(){  
    ...
```

Por que eu usaria isso?

A princípio, as filas e pilhas parecem ser simplesmente vetores limitados. Isso é verdade até certo ponto. Existe, inclusive, uma forma simples de simular uma pilha/fila usando vetores comuns (você vão ter que fazer isso em Estruturas de Dados, no segundo semestre). Então, surge a pergunta, pra que serve isso?

A resposta é conveniência. Existem problemas ou situações que seguem literalmente o comportamento de uma pilha ou uma fila. Nessas situações, embora o uso de um vetor fosse perfeitamente possível, fica muito conveniente (e bizarramente menos trabalhoso) usar filas ou pilhas adequadamente.